

Industrial Project Report

Flexible Platform for Multi-Stage Attack Scenario Orchestration

Program	SNE25-26 · Master's in Security of Systems and Networks
Course	Industrial Project
Client	Innostage
Team members	Nikita Niakhai · Mark Chausov · Hasib Al Tahsin · Khalid Maina · Albert Avkhadeev · Alisa Evdosenko · Victor Adekanye
Project Repository	https://github.com/lcensies/sliver-orchestrator/
Video demo	Flexible Platform Demo.mp4 (https://drive.google.com/file/d/1JpfDs3DTqwe25P4ButZVdoxgps9EVbXY/view?usp=sharing)
Documentation	README.md, Industrial Project Report
Date	25.07.2026

INNOPOLIS UNIVERSITY

1.0 INTRODUCTION

1.1. Project Context

This report documents an industrial project undertaken in the security of systems and networks department. The goal of the industrial project was to develop a flexible platform constructor where a mentor can build complex scenarios (Initial Access → Execution → Persistence → Lateral Movement → Impact) from modules and defined templates and then execute them on the infrastructure without requiring a low-level operation of the tools used. The project was completed as part of the Industrial Project course in the Master Security of Systems and Networks programme.

1.2. Goals and Objectives

The aim was to deliver a working solution comprising both frontend and backend components with the following objectives:

- I. Develop a backend service capable of orchestrating multi-stage attack scenarios on cyber-range infrastructure;
- II. Develop a frontend web application through which mentors can design, execute, and monitor scenarios;
- III. Integrate industry-standard tooling—specifically the Sliver command-and-control framework and the MITRE ATT&CK Atomic Red Team technique library—as the foundation for scenario execution
- IV. Package the solution so that a complete lab environment can be reproduced on a single host with minimal manual intervention;
- V. Test and document the resulting system to a level appropriate for handover.

The project was organized as a distributed team, with coordination performed asynchronously through Notion (task and requirement tracking), Telegram (day-to-day communication), and regular meetings hosted on Kontur. English was the shared working language.

1.3. Results of the Previous Semester

The spring semester of the project concluded with a MVP. By the end of that semester, the team had delivered the following components, each functional

- I. Go backend service (the scenario server) exposing a REST API for creating, storing, and executing attack chains. The backend integrated with the Sliver C2 framework through its gRPC operator interface, wrapped the Atomic Red Team YAML technique library, and persisted state to SQLite. It supported five action types (command, atomic, upload,

binary, `sliver_rpc`); a directed-acyclic-graph execution model with topological ordering and cycle detection; and Server-Sent Events for streaming execution updates.

- II. A React and TypeScript frontend (the flexible platform) providing a visual scenario editor, a MITRE ATT&CK atoms browser, a live sessions view, and an execution viewer that consumed the Server-Sent Events stream. The frontend used Mantine for its component library, Redux Toolkit with RTK Query for state and API management, and React Flow for the dependency graph visualization.
- III. A Docker-based lab environment that provisioned a Sliver C2 server, a scenario-server instance, and a Linux victim beacon in a single Docker Compose invocation.
- IV. An initial catalogue of demonstration scenarios in YAML, covering basic discovery techniques on a Linux target.

1.4 Objectives for This Semester

Building on the spring semester's MVP, the summer semester was organized around five objectives, each assigned to specific team members. Main objectives include:

- I. Extend the backend with additional features required for realistic multi-stage scenarios, including per-step session targeting for multi-victim chains and improvements to the payload-delivery pipeline, compatibility with initial access without a dedicated session on a victim machine.
- II. Extend the lab environment to Windows targets and enable lateral movement scenarios, together with the credential-access and persistence techniques that are relevant to enterprise-style attack chains.
- III. Establish comprehensive test coverage on both the backend and the frontend, including unit tests and integration tests.
- IV. Merge the parallel feature streams into a single stable integration branch; resolve the build and networking issues that surfaced during integration; restructure the repository so that frontend and backend could be cloned and deployed together; and produce reproducible demonstration scenarios.
- V. Align the frontend with the extended backend capabilities in particular, the per-step session override, and finalize the user-facing workflow.

1.5 Report Structure

The remainder of this report follows the IMRAD structure. Chapter 2 (Methods) describes the tools and technologies used this semester, the development methodology adopted by the distributed team, and the timeline of work. Chapter 3 (Results) is organised by personal contribution and presents each team member's work during the summer semester in a dedicated subsection. Chapter 4 (Discussion) reflects on the outcome, identifies the limitations of the

current system, and captures the lessons learned during integration. Chapter 5 (Conclusion) summarises the state of the platform at the end of the semester. Deployment guides, full scenario listings, architectural diagrams, and additional technical detail are provided in the appendices, which do not count towards the main-body page limit. Appendix includes: screenshots of UI (DoD), yaml scenario for final initial access chain, logs of the executed scenario.

2.0 METHODS

2.1. Tools and Technologies Used

Development tools

- I. **Frontend:** React 18, TypeScript, Vite, Mantine, Redux Toolkit, RTK Query
- II. **Backend:** Go, REST API, SSE, Sliver gRPC integration
- III. **Database:** SQLite with GORM
- IV. **API:** REST /api/v1, JSON/YAML, Server-Sent Events for live execution logs
- V. **Infrastructure:** Sliver C2, Vagrant, VirtualBox, Linux Victim and Windows target, Docker compose
- VI. **Version control:** Git, 2 public GitHub repositories
- VII. **Testing:** Vitest/React Testing Library on frontend, Go tests on backend

Project management and communication

- I. **CRM / task management system:** Notion.
- II. **Communication:** Telegram group chats, Kontur and Yandex Telemost online meetings.
- III. **Documentation:** README files, GIT history, present reports and design models of system.

2.2 Functionality

As illustrated in Figure 1, the platform is delivered as two integrated components: a Go orchestration backend (sliver-orchestrator) and a React web UI (flexible-platform) deployed against two complementary lab environments with a full attack scenario run against them.

2.2.1 Backend

The backend is a Go service that wraps Sliver's gRPC operator API and exposes a REST interface over which multi-stage attack chains are defined, validated, and executed against active implant sessions.

Chain Execution Engine

Attack chains are modelled as directed acyclic graphs (DAGs) of steps. A topological resolver validates the graph, rejects cycles or missing dependencies, and produces an execution order.

Atomic Red Team Integration

The backend loads 340 MITRE ATT&CK atomic techniques from the Atomic Red Team YAML library on start. Techniques are resolvable by GUID, name, or test index; user-supplied arguments are interpolated into the test's command template before dispatch. Making it easy for mentors to carry out attack scenarios following the framework

2.2.2 Lab Environments

Two lab deployments were produced this semester. The Vagrant multi-hop lab, added this semester, provides a realistic multi-victim topology capable of executing a full lateral-movement and credential-access chain. The Docker lab, extended from the previous semester's MVP, provides a fast single-host environment used for backend integration testing and end-to-end validation of the chain executor and frontend.

Vagrant multi-hop lab

The Vagrant lab is a nested-virtualisation topology hosted on a Kali workstation inside VMware, with three VirtualBox VMs orchestrated by Vagrant. The topology reproduces the network segmentation typical of a real target environment: the attacker's C2 sits on a management network, the Linux victim acts as a pivot with a foot in both the management network and an isolated internal network, and the Windows target is reachable only from the pivot via the internal network.

2.3.2 Frontend

As illustrated in Figure 1 the frontend is a React 18 and TypeScript single-page application built with Vite, Mantine, Redux Toolkit with RTK Query, and React Flow. The DAG viewer performs cycle detection and missing-dependency checks in the browser before the chain is sent to the backend; heavy dependencies (xterm.js, CodeMirror, React Flow) are loaded to keep using `react.lazy()` to keep the bundle small. The whole interface was also configured to be bilingual (English and Russian) via *il8next*

2.3 Spring Semester Timeline

The spring semester established the project's foundations. The team formed, the client's requirements were captured, an initial architecture was proposed, and a working minimum viable product was delivered by the end of the semester. Seven stages summarise the spring semester's work:

Stage	Description	Period (2026)	Responsible
0. Initial Analysis	Understanding the task, requirements, diving into the context of managing initiative team management tasks. Creating UML models for the system	25.02 - 02.03	Nikita, Alisa, Khalid, Hasib
1. Technologies review and design proposition	Survey of C2 frameworks and workflow engines; architecture proposal	02.03 - 06.03	Mark, Khalid
2. Development of the main functionality for the backend	Chain executor, REST API, Sliver gRPC integration, SQLite store	06.03 - 18.03	Mark
3. Development of the frontend	React app development	18.03 - 31.03	Nikita
4. Additional backend features	Developing and testing new required features, fixing bugs	18.03 - 16.04	Khalid, Hasib, Albert
5. Merging feature branches and finalizing backend	Tested the Docker lab environment end-to-end with C2 and victim containers	20.04 - 10.05	Victor
6. Aligning frontend with new backend functionality	Deployed and integrated both the front-end and back-end	10.05 - 18.05	Victor, Nikita
7. Testing and bug fixing	Created tests for all system components, discovered issues in backend implementation and fixed it	11.05 - 18.05	Alisa
8. Finalizing the report	Preparing demo, repository, and final report	18.05 - 20.05	Nikita, Victor, Hasib, Mark, Khalid, Alisa

Table 1. Spring Semester Timeline

2.4 Summer Semester Timeline

The summer semester extended the MVP into a defensible working solution. Six parallel workstreams ran through June and were consolidated into a single integrated system in July.

Stage	Description	Period (2026)	Owner
Automated payload delivery	lateral movement examples, making scenarios executable independently from the session id	01.06 - 15.06	Khalid, Hasib
Lateral movement, Windows alignment. Vagrant multi-hop lab	Three-VM topology (C2, Linux pivot, Windows target), autoboot pipeline, honeypot, fourteen-step attack chain	16.06 - 01.07	Khalid, Hasib, Mark
Building of the Final chain scenario	Creating automated steps. Making initial access and payload delivery automated. Building and testing yaml file	04.07 - 20.07	Hasib, Khalid, Mark
Integration and finalisation	Migration to Vagrant also integrating frontend as a submodule of the backend repository, produced reproducible demonstration scenarios	15.07 - 20.07	Nikita, Hasib
Test coverage and bug fixes	Integration testing, Go tests on the backend, defect triage	24.07 - 26.07	Khalid, Hasib, Mark, Alisa
Report, demo, defence	Finalising the report, recording the demo, preparing and rehearsing the presentation. Cleaning github repositories, branches	26.07 - 30.07	Victor, Khalid, Hasib, Nikita

Table 2. Summer Semester Timeline

3.0. RESULTS

The results gotten this summer semester delivered a working solution following the client request. The following was achieved.

3.1 Flagship Attack Chain

The main functional deliverable is a fourteen-step attack chain (*examples/full-attack-chain-v2.yaml*) that models the post-initial-access half of a realistic multi-stage intrusion. The chain assumes an active Sliver session on the Linux pivot, which is obtained separately through the honeypot exploitation path [Initial access module] and proceeds through discovery, persistence, lateral movement, and credential access on a Windows target. Every step is tagged with the corresponding MITRE ATT&CK technique identifier so the chain is traceable against a standard adversary model.

#	Step ID	MITRE	Type	Action
1	linux_sysinfo	T1082	atomic	OS and kernel enumeration on the pivot
2	linux_network	T1016	atomic	Network configuration discovery
3	T1016_2_mrxv767j_2	T1016	atomic	Duplicate T1016 (UI-authoring artefact, slated for removal)
4	cron_persistence	T1053.003	atomic	Plant cron backdoor on the pivot
5	systemd_persistence	T1543.002	atomic	Create persistent systemd service
6	ping_sweep	T1018	command	Parallel ping sweep of 172.16.1.0/24
7	reliable_scan	T1046	command	TCP port check for SSH, SMB, WinRM on 172.16.1.20
8	test_defaults	T1078	command	Try four default credential pairs via wmiexec.py
9	lateral_recon	T1021.006	command	Impacket WMIExec – hostname on Windows target
10	win_sysinfo	T1082	command	Confirm Windows hostname via WMIExec
11	deploy_implant	T1105	command	Trigger WindowsUpdateHelper scheduled task via WMI
12	sam_dump	T1003.002	command	Impacket secretsdump.py – SAM and LSA extraction

#	Step ID	MITRE	Type	Action
13	find_dc_candidate	T1018	command	Scan Kerberos, LDAP, SMB ports across candidate IPs
14	shadow_copy_vs	T1003.003	command	Trigger VSSDump scheduled task via WMI

Table 3 Attack Scenarios Methodology

In the Implementation we had some design points. First, the split between atomic-typed and command-typed steps is deliberate. Well-defined discovery and persistence techniques (steps 1–5) execute genuine MITRE Atomic Red Team YAML tests by identifier, inheriting the community-maintained taxonomy. The multi-step lateral-movement and credential-access flow (steps 6–14) required composing Impacket calls in a specific order for which no single atomic exists, so these steps are inline shell commands tagged with the closest MITRE identifier for traceability.

Second, Sliver acts as the transport and session manager while Impacket provides the technique tooling. Each step's shell command is dispatched by the backend through Sliver's Execute gRPC to the pivot's Sliver beacon; the pivot then invokes Impacket utilities (*wmiexec.py* and *secretsdump.py*) to perform the actual WMI operations against the Windows target.

The chain runs to completion in approximately 60 seconds against a warm Vagrant lab. Step 12 (*sam_dump*) produces live NTLM password hashes captured from the target's SAM database – evidence which is normally simulated in training scenarios and is reproducible in this lab from a clean boot. A representative capture is included as *Appendix A*. A smaller catalogue of single-purpose scenarios (single-technique discovery, WinRM credential spray, two-beacon session hand-off, ICMP reachability probe, and others) is bundled alongside the flagship chain in the *examples/* directory. These serve both as regression tests for the executor and as reusable building blocks that mentors can compose into custom scenarios.

3.2 Testing and Bug Fixes

We combined automated unit tests, end-to-end runs against the Vagrant lab, and integration testing against the Docker lab. Notable defects discovered and fixed during this cycle includes; atomics path resolution failing when scenario-runner was invoked from a non-standard working directory, and chain execution failing in an isolated repository environment. Bug fixes were verified by re-running the affected scenarios.

4.0 DISCUSSION

The summer semester delivered a working solution against the client's original brief. Three claims can be made and defended by the artefacts produced. First, the platform allows a mentor

to design multi-stage attack scenarios in a browser and execute them against live victims on demand with desired functional of payload delivery, lateral movement; the flagship fourteen-step chain runs end-to-end from a single click, without any operator input at the C2 command line. Second, the platform now supports both Linux and Windows targets in the same chain: lateral movement from a Linux pivot into a Windows target is a first-class scenario type, not a demonstration outside the tool. Third, the entire lab is reproducible from a clean checkout: the Docker environment reaches a working state in under two minutes, and the Vagrant multi-hop environment recovers automatically across Kali reboots through a systemd-orchestrated boot sequence.

These results address the two gaps left open at the end of the spring semester. The end-to-end integration is now demonstrable through the frontend-and-backend clone-and-run flow, backed by a git submodule that pins the frontend to a known-good commit inside the backend repository. The Windows and lateral-movement coverage gap is closed by the */implant/windows* endpoint, the per-step session_id override in the chain schema, the Vagrant three-VM topology, and the flagship chain that exercises all three simultaneously.

4.1 Lessons Learned

During the Project we learned:

Full-Stack development

- Go Development based on sliver
- Lateral movement Attacks using MITRE ATT&CK framework
- Automated Payload delivery and initial access techniques
- Difference between implementing scenarios on Windows and Linux
- Delivering working red-blue team solutions
- Defense and Security techniques.

Communication

- Multicultural team, different work and ethics approaches, building communication bridges
- Basics in team management and effective communication.

4.2 Limitations

- I. For storage persistence, SQLite was used to store chains, executions and logs for a simple one-mentor lab environment. For a much bigger scale the project would need to be migrated to a much larger database.
- II. Authentication and Authorization methods. MAC, RBAC for accessing the system.
- III. Integration with other platform and tools.

4.3 Future Work

- I. Auto-detects the target platform for atomic steps, so for the project the scenario author supplies only the technique ID and the executor selects the appropriate test variant based on the resolved session's OS.
- II. Migrate persistence to PostgreSQL behind a driver flag, keeping SQLite as the single-host default. GORM makes this a configuration change; the schema itself does not need to move.
- III. Add bearer-token authentication at the API layer and a corresponding login screen in the frontend, opening the path to multi-tenant classroom deployment.
- IV. Parameterise Windows persistence artefacts (scheduled task names, registry keys, implant URL, target user account) so that the Vagrant lab's flagship chain is portable to other Windows targets without editing the chain YAML.
- V. Add a scenario-runner CLI as a proper Go binary (replacing the current `lab-run.py` Python prototype), so mentors can run chains from a shell script and integrate the platform into CI or scheduled training runs.

4.4. Conclusion

This semester extended the platform from a spring-semester MVP into a working, defensible solution that meets the client's brief: a mentor can now design and execute a multi-stage attack scenario against a reproducible cyber-range environment through a single browser tab, without operating the underlying C2 by hand. The delivered system covers the full attack life-cycle including Windows targets and lateral movement, and its two lab deployments – the fast Docker environment and the Vagrant multi-hop topology which support both routine development and the flagship fourteen-step demonstration chain. The limitations and future work identified in Chapter 4 outline concrete next steps for a subsequent development cycle, but the platform in its current state is ready for hand-over to the client and for use in operator training.

APPENDIX

Architecture of the Project

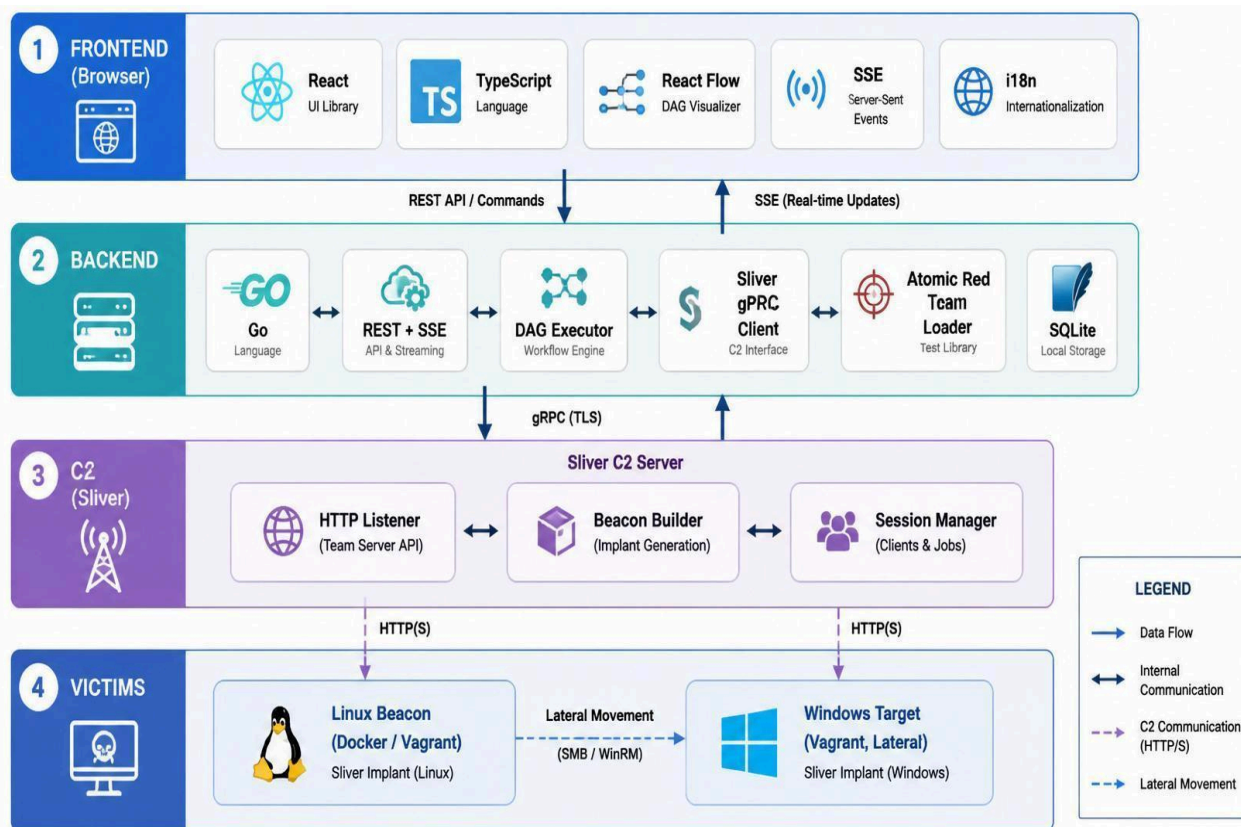


Figure 1. Architecture of the Flexible Platform (high-level)

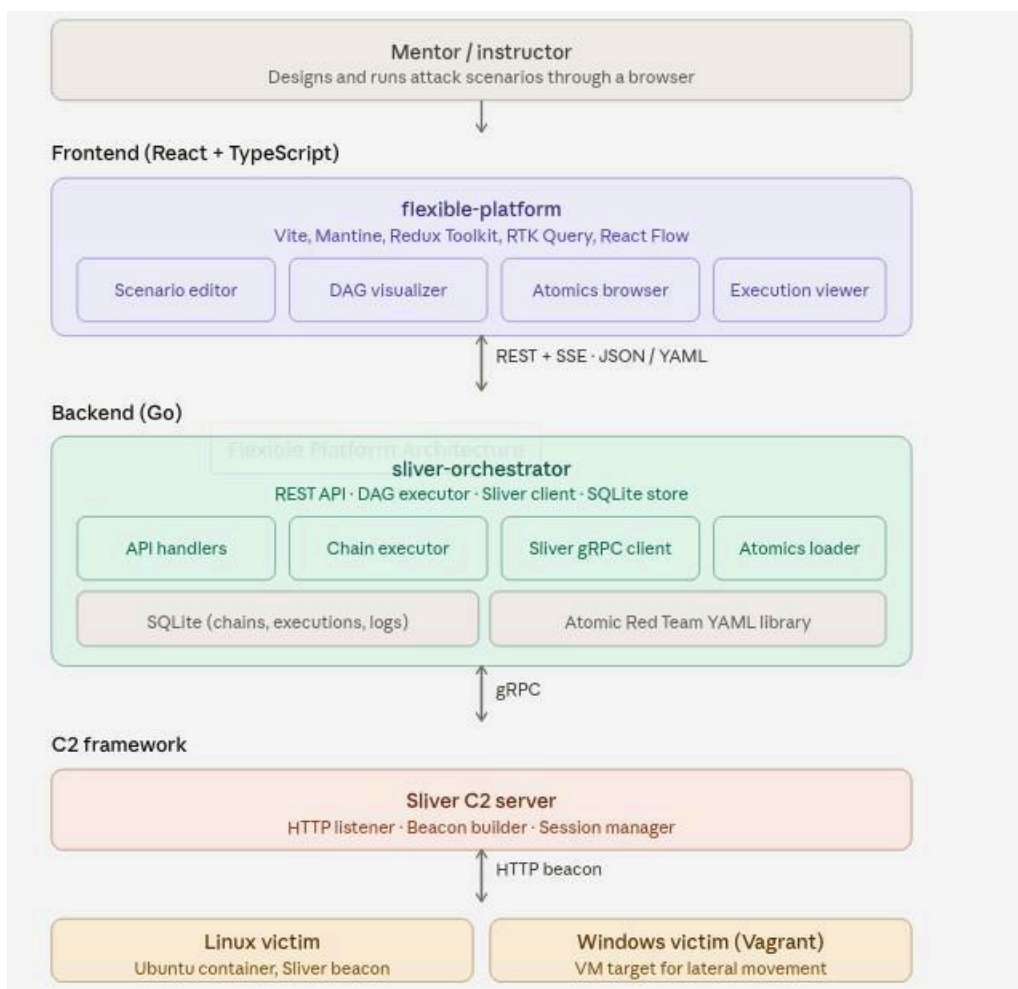


Figure 2. Component architecture

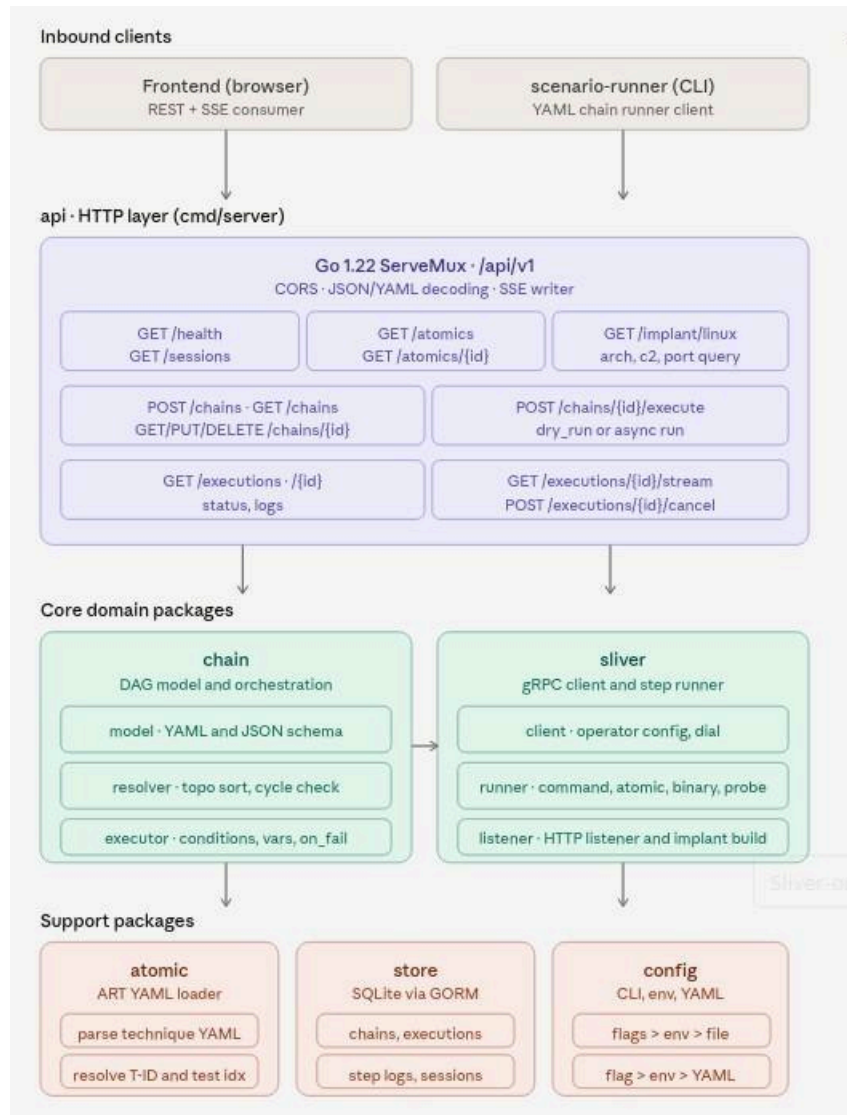


Figure 3. Backend architecture

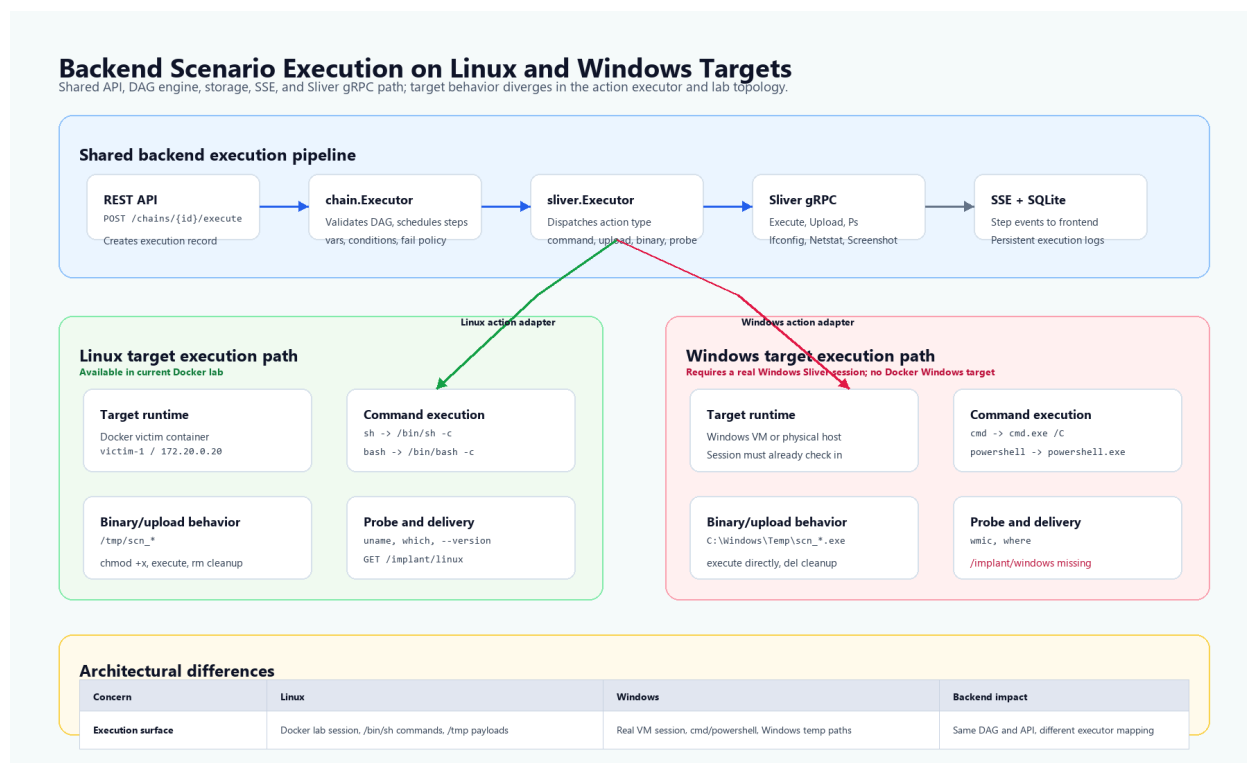


Figure 4. Linux & Windows execution architecture

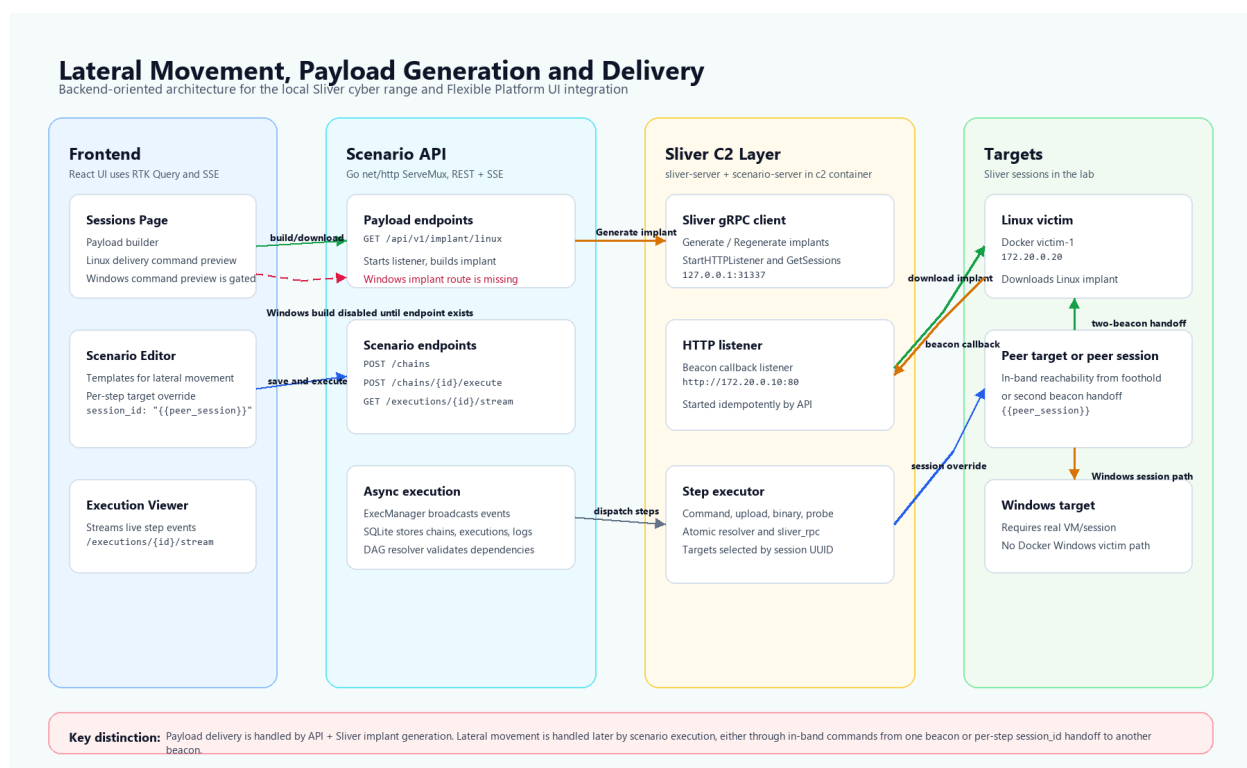


Figure 5. Lateral movement payload delivery

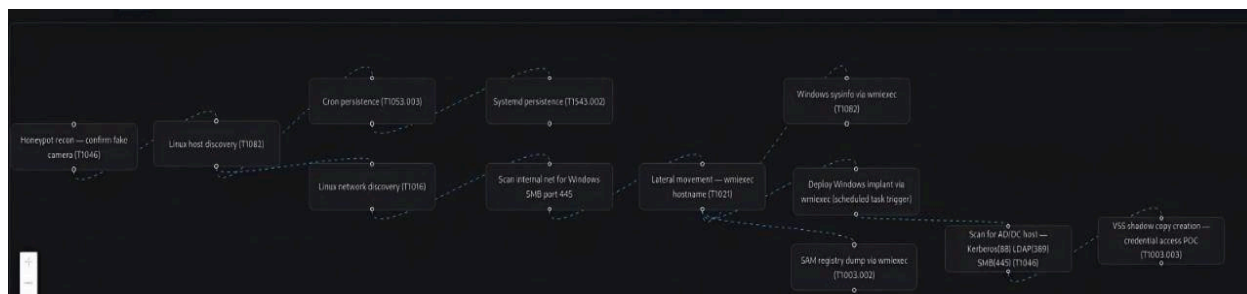


Figure 6. DAG Attack chain

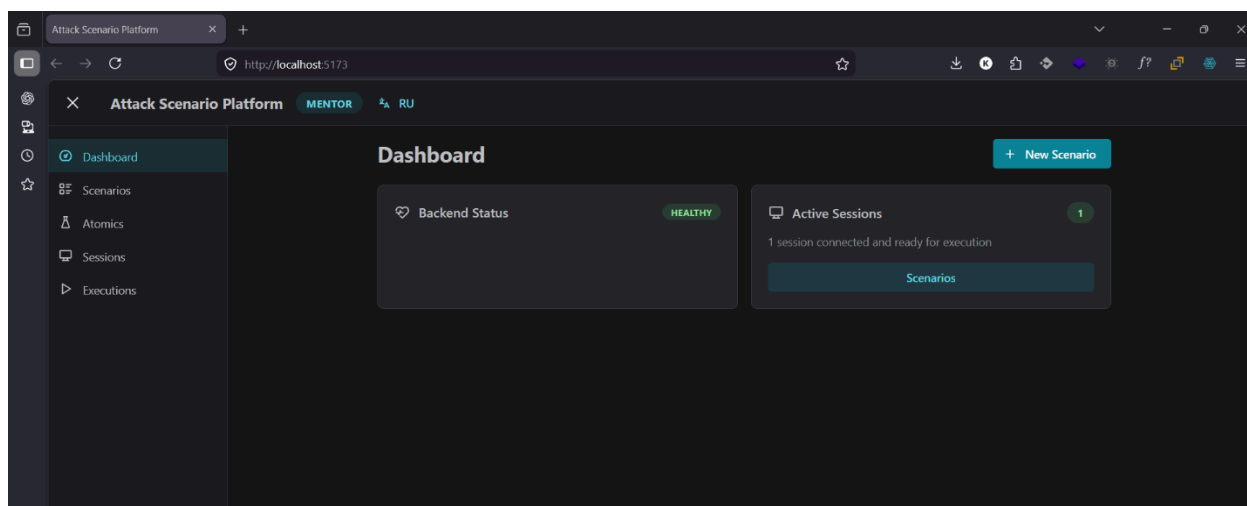


Figure 7. UI homepage

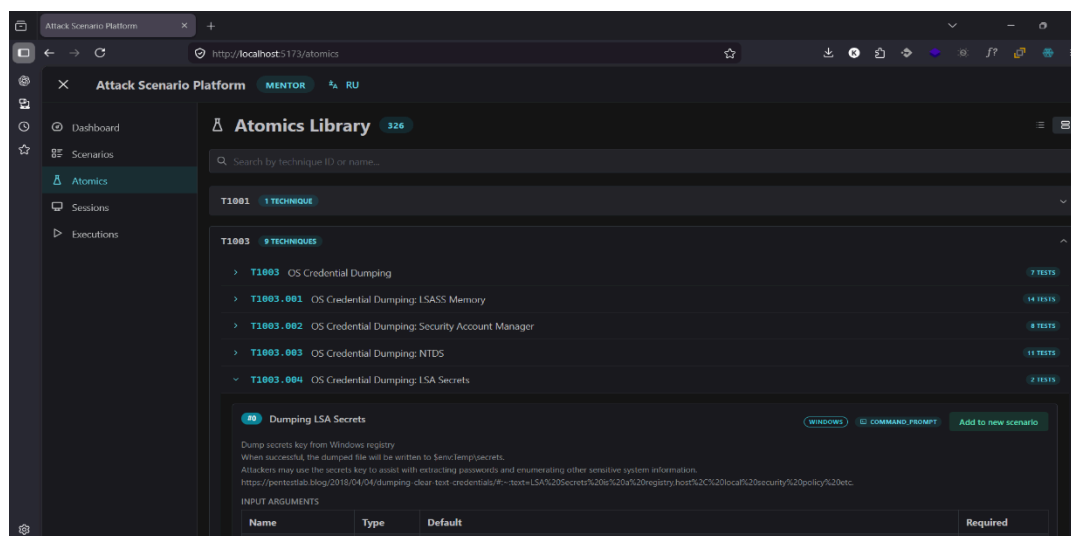


Figure 8. Atomic library with Mitre Att&ck framework

#	Step ID	Name	Action Type	Depends On	On Fail	Actions
#	1	linux_sysinfo	Linux host discovery (T1082)	ATOMIC RED TEAM	—	CONTINUE
#	2	linux_network	Linux network discovery (T1016)	ATOMIC RED TEAM	linux_sysinfo	CONTINUE
#	3	T1016_2_mrxv767j_2	T1016 — System Network Configuration Discovery	ATOMIC RED TEAM	linux_network	CONTINUE
#	4	cron_persistence	Cron persistence (T1053.003)	ATOMIC RED TEAM	T1016_2_mrxv767j_2	CONTINUE
#	5	systemd_persistence	Systemd persistence (T1543.002)	ATOMIC RED TEAM	cron_persistence	CONTINUE
#	6	ping_sweep	Parallel ping sweep	COMMAND	systemd_persistence	CONTINUE
#	7	reliable_scan	Scan SMB, SSH, WinRM (2s timeout)	COMMAND	ping_sweep	CONTINUE
#	8	test_defaults	Test multiple default credentials	COMMAND	reliable_scan	CONTINUE
#	9	lateral_recon	Lateral movement — wmiexec hostname (T1021)	COMMAND	test_defaults	CONTINUE
#	10	win_sysinfo	Windows sysinfo via wmiexec (T1082)	COMMAND	lateral_recon	CONTINUE
#	11	deploy_implant	Deploy Windows implant via wmiexec (scheduled task trigger)	COMMAND	win_sysinfo	CONTINUE
#	12	sam_dump	SAM registry dump via wmiexec (T1003.002)	COMMAND	deploy_implant	CONTINUE_NO_ERR
#	13	find_dc_candidate	Scan for AD/DC host — Kerberos(88) LDAP(389) SMB(445) (T1046)	COMMAND	sam_dump	CONTINUE_NO_ERR
#	14	shadow_copy_vss	VSS shadow copy creation — credential access POC (T1003.003)	COMMAND	find_dc_candidate	CONTINUE_NO_ERR

Figure 8. 14-step scenario implementation

Full Scenario Logs

#	1	exploit_vulnweb	Exploit ResolvTech command injection + deploy beacon (T1190)	PYTHON SCRIPT	—
#	2	confirm_foothold	Confirm foothold — id + hostname + uname (T1082)	COMMAND	EXPLOIT_VULNWEB
#	3	network_discovery	Network config + routing discovery (T1016)	COMMAND	CONFIRM_FOOTHOLD
#	4	cron_persistence	Cron persistence (T1053.003)	COMMAND	CONFIRM_FOOTHOLD
#	5	systemd_persistence	Systemd service persistence (T1543.002)	ATOMIC RED TEAM	CONFIRM_FOOTHOLD
#	6	bashrc_persistence	Bashrc persistence (T1546.004)	COMMAND	CONFIRM_FOOTHOLD
#	7	find_windows_host	Dynamic Windows host discovery — nmap/arp-scan/TCP sweep (T1046)	COMMAND	NETWORK_DISCOVERY
#	8	bruteforce_credentials	Bruteforce default credentials via WMIExec (T1110.001)	COMMAND	FIND_WINDOWS_HOST
#	9	lateral_movement	Lateral movement via WMIExec (T1021.006)	COMMAND	FIND_WINDOWS_HOST
#	10	win_sysinfo	Windows system info via WMIExec (T1082)	COMMAND	LATERAL_MOVEMENT
#	11	win_network	Windows network config via WMIExec (T1016)	COMMAND	LATERAL_MOVEMENT
#	12	deploy_windows_implant	Deploy Sliver beacon on Windows via schtasks (T1105)	COMMAND	LATERAL_MOVEMENT
#	13	credential_dump	SAM credential dump via secretdump (T1003.002)	COMMAND	LATERAL_MOVEMENT
#	14	find_dc_candidate	DC candidate discovery — Kerberos/LDAP/SMB/GC (T1018)	COMMAND	CREDENTIAL_DUMP
#	15	shadow_copy_vss	VSS shadow copy creation POC (T1003.003)	COMMAND	FIND_DC_CANDIDATE

Figure 9. Full Scenario chain scenarios

```

Logs: win_network DONE

Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
IPv4 Address. . . . . : 10.0.2.15(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::2%15
Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter #2
IPv4 Address. . . . . : 172.16.1.20(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :
Description . . . . . : Intel(R) PRO/1000 MT Desktop Adapter #3
IPv4 Address. . . . . : 192.168.56.20(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :

```

Figure 10. Windows Network

```

Logs: exploit_vulnweb  DONE

[*] Target: http://192.168.56.10:8080
[*] Testing command injection (T1190 – Exploit Public-Facing Application) ...
[+] RCE confirmed: uid=0(root) gid=0(root) groups=0(root)
[+] Injected: nohup+/tmp/.svc+>/dev/null+2>81+%26 ...
[+] INITIAL_ACCESS_PAYLOAD_DELIVERED
[*] Waiting for beacon callback ...
[*] Session found: 72c13370-747a-4409-b385-501ef3725949
[*] Waiting 15s for session to stabilize ...
CAPTURED_SESSION: 72c13370-747a-4409-b385-501ef3725949

```

Figure 11. Windows Network

```

Logs: find_dc_candidate  DONE

=== Scanning for DC candidate ===
DC_CANDIDATE: 172.16.1.20 port:445 OPEN

```

Figure 12. Scanning

```

Logs: network_discovery  DONE

3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5b:15:04 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe5b:1504/64 scope link
        valid_lft forever preferred_lft forever
4: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:81:cc:13 brd ff:ff:ff:ff:ff:ff
    inet 172.16.1.10/24 brd 172.16.1.255 scope global enp0s9
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe81:cc13/64 scope link
        valid_lft forever preferred_lft forever
[*] Routing table:
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
172.16.1.0/24 dev enp0s9 proto kernel scope link src 172.16.1.10

```

Figure 13. Network discovery

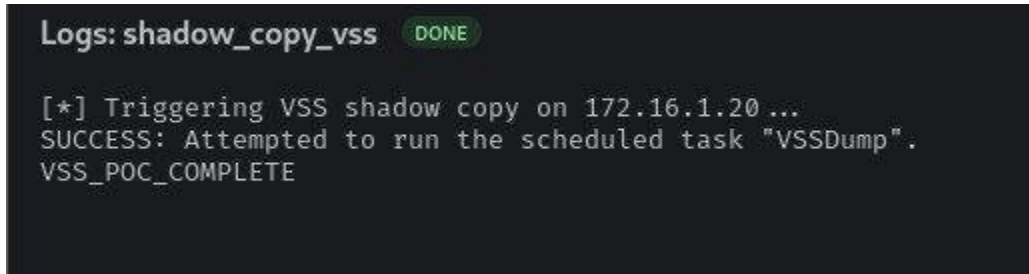
```

Logs: credential_dump  DONE

[*] Dumping SAM hashes from 172.16.1.20 via secretsdump...
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:4cbb09cc75f3b75e9c8606b82a769d03:::
vagrant:1001:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f71d913c245d35b50b:::
CREDENTIAL_DUMP_COMPLETE

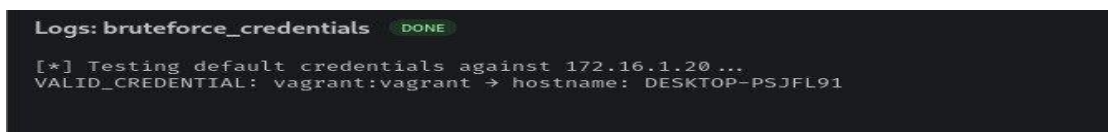
```

Figure 14. SAM hashes dumped

A terminal window with a dark background. The title bar reads "Logs: shadow_copy_vss" followed by a green "DONE" button. The output text is as follows:

```
[*] Triggering VSS shadow copy on 172.16.1.20 ...  
SUCCESS: Attempted to run the scheduled task "VSSDump".  
VSS_POC_COMPLETE
```

Figure 15. SAM hashes dumped

A terminal window with a dark background. The title bar reads "Logs: bruteforce_credentials" followed by a green "DONE" button. The output text is as follows:

```
[*] Testing default credentials against 172.16.1.20 ...  
VALID_CREDENTIAL: vagrant:vagrant → hostname: DESKTOP-PSJFL91
```

Figure 16. SAM hashes dumped

Team Composition and Roles

Team Member	Skills	Impact	Grade Recommendation
Mark Chausov	System programming, backend development, red/blue teams, the MITRE ATT&CK framework and Atomic Red Team. Exploit development, CTFs	Developed whole backend. Great understanding of context. High level of skills and efficiency, contributed by hosting the project repository, also helped in Bug troubleshooting and Fixing	A
Nikita Niakhai	Front-end development, Team management communication	Developed Front end. Managed team and lead communication with the client. Divided roles and responsibilities in the team. Delivered objectives to Foreign team members in English (calls, AI translation and transcriptions)	A
Hasib Al Tahsin	Linux System Administration, ML , Networking	Transferred the setup entirely to Vagrant, added a Windows VM as a second victim and pivoted the first victim to access the Windows VM. Bug troubleshooting and Fixing	A
Khalid Maina	GO developer, Red/ blue team, mitre attack framework, Atomic red team, Exploit and payload development	Developed functionality of implant, carrying out the lateral movement of attack on the victim, Bug troubleshooting and Fixing	A
Victor Adekanye	System Administration, Networking, Version control, MLOps	Performed sliver-orchestrator manual testing, also created YAML scenarios inside UI from the red teamer perspective assisting in Troubleshooting and Fixing Bugs	A
Albert Avkhadeev	Python, GO development.	Developed functionality for lateral movement and payload delivery	B
Alisa Evdosenko	Deployed and integrated both the front-end and back-end	Created comprehensive automated unit/integration/black-box end-to-end	B